

JIRA Defect Lifecycle Intelligence Platform

Version 4.0

Knowledge Governance & Organizational Learning Architecture

1. Executive Summary

Version 4.0 represents the transition from Case Intelligence to Knowledge Governance.

Previous generations focused on:

- v1.0 Workflow Intelligence
- v2.0 Trustworthy Intelligence
- v3.0 Case Intelligence

Version 4.0 introduces organizational memory, ontology governance, human feedback learning, and architecture inheritance.

The system is no longer limited to analyzing individual defects.

It becomes a continuously evolving engineering knowledge platform.

2. Core Design Principles

Principle 1 — Fact Before Interpretation

Facts must always be extracted before compliance evaluation and recommendation generation.

No Agent may modify or reinterpret Facts generated by Fact Agent.

Principle 2 — Rules Before Reasoning

Compliance decisions are produced by Rule Engine.

LLMs may explain decisions but may never generate compliance decisions independently.

Principle 3 — Human Authority Supremacy

Human reviewers remain the final authority.

All AI recommendations are advisory.

Principle 4 — Knowledge Must Evolve

Historical decisions, architectural patterns, and governance knowledge must continuously improve the platform.

Principle 5 — Ontology Is the Single Source of Truth

All reusable knowledge must be represented using canonical ontology entities.

Free-text knowledge is considered transient and non-governable.

3. High-Level Architecture

Raw Sources

- Jira
- XML Exports
- Changelogs
- Attachments
- Telemetry Logs

↓

Shared Infrastructure Layer

- MCP Tools
- MCP Resources
- Skills Engine

↓

Fact Agent

↓

Review Agent

↓

Advisor Agent

↓

Human Decision Layer

↓

Knowledge Governance Layer

↓

Knowledge Store

4. Agent Architecture

4.1 Fact Agent

Responsibilities:

- Timeline reconstruction
- Clone topology reconstruction
- Evidence extraction
- TCI calculation
- PFI calculation
- Historical pattern retrieval

Outputs:

Fact JSON

The Fact Agent is prohibited from:

- Risk grading
 - Limitation eligibility decisions
 - Compliance decisions
-

4.2 Review Agent

Responsibilities:

- Compliance verification
- Limitation eligibility evaluation
- Security policy enforcement
- Evidence completeness validation

Supported Decisions:

- MUST_FIX_BLOCKER
- TEMP_LIMITATION_ELIGIBLE
- CRITICAL_SSRB_REVIEW
- INVALID_CLOSURE_REQUEST
- INSUFFICIENT_EVIDENCE

The Review Agent operates exclusively through Rule Engine execution.

4.3 Advisor Agent

Responsibilities:

- Action generation
- Risk communication
- Executive summaries
- TIP drafting

Inputs:

- Fact JSON
- Review JSON

Outputs:

Three-track recommendation matrix:

- Preferred Path
- Alternative Path
- Escalation Path

Advisor Agent may not alter Review Agent decisions.

5. Case Intelligence Layer

5.1 Purpose

Retrieve historical engineering experience.

Convert historical defect resolution patterns into actionable context.

5.2 Retrieval Strategy

Primary Match Dimensions:

- Technical Domain
 - Affected Component
 - Failure Signature
 - Root Cause Category
 - Platform Family
-

5.3 Similarity Confidence

Cases are categorized:

- Strong Match (>90%)
- Related Match (70–90%)
- Reference Match (50–70%)

Below 50% similarity is discarded.

6. Knowledge Governance Layer

Version 4.0 introduces governance as a first-class system component.

7. Ontology Governance Framework

7.1 Problem

New technologies continuously emerge:

Examples:

- CXL
- PFR 3.0
- New BMC architectures
- Future platform families

Static ontology eventually becomes obsolete.

7.2 Ontology Shadow Suggestion

When canonicalization fails:

Agent may create:

Draft_Ontology_Node

Example:

Draft: SPI_Timing_Jitter

Status: PENDING_REVIEW

Confidence: 0.87

Source Cases: 23

7.3 Human Promotion Workflow

Draft Node

↓

Architecture Review Board

↓

Approve / Reject

↓

Ontology Update

Only humans may promote ontology nodes.

Agents may suggest.

Agents may never publish ontology changes.

8. Human Feedback Flywheel

8.1 Purpose

Capture organizational learning.

Prevent repeated recommendation mistakes.

8.2 Override Event

When human reviewers override AI recommendations:

Override Event must be recorded.

Example:

System Decision: TEMP_LIMITATION

Human Decision: MUST_FIX

Reason: Customer-visible availability impact

Reviewer: Senior PE

Timestamp: 2026-06-15

8.3 Correction Pattern Generation

Override events are transformed into:

Correction Patterns

Example:

Pattern: Customer-visible KVM latency

Historical Outcome: 82% escalated to Must Fix

Confidence: 0.91

8.4 Knowledge Store Integration

Correction Patterns become searchable knowledge artifacts.

Future Advisor generations must consider these patterns.

9. Architecture Inheritance Engine

9.1 Problem

New platforms frequently have no historical cases.

Traditional retrieval returns zero results.

9.2 Inheritance Graph

Example:

Platform_GNR

↓

Platform_BHS

↓

Platform_Next

Inheritance Weight: 75%

9.3 Ancestor Retrieval

When no local cases exist:

Retrieve:

- Parent Platform Cases
 - Shared Component Cases
 - Shared Technical Domain Cases
-

9.4 Generalized Knowledge Transfer

Historical defects may be reused when:

Shared Component Similarity > 70%

or

Architecture Inheritance Weight > 60%

10. Confidence Escalation Framework

Version 4.0 replaces Dual-Model Consensus.

10.1 Confidence Categories

High Confidence

0.90–1.00

Medium Confidence

0.75–0.89

Low Confidence

<0.75

10.2 Escalation Rules

Low confidence canonicalization:

→ Human Review

Low confidence ontology match:

→ Draft Ontology Queue

Low confidence recommendation:

→ Advisory Warning

11. Knowledge Store Architecture

Knowledge Store contains only structured entities.

Supported Objects:

- Canonical Defect Models
- Ontology Entities
- Correction Patterns
- Architecture Graphs
- Historical Cases
- Governance Decisions

Raw Jira comments are not stored as knowledge.

12. Knowledge Lifecycle

Knowledge Creation

↓

Canonicalization

↓

Validation

↓

Human Governance

↓

Promotion

↓

Knowledge Store

↓

Case Retrieval

↓

Advisor Consumption

↓

Human Override

↓

Feedback Flywheel

↓

Knowledge Evolution

13. Auditability Requirements

Every recommendation must be traceable.

Required Trace Links:

- Fact Evidence
- Review Decision
- Historical Cases
- Correction Patterns
- Ontology Entities

All generated reports must include Pipeline Fingerprint metadata.

14. Roadmap

Version 5.0 Target:

Engineering Digital Twin

Planned Capabilities:

- Cross-program defect prediction
 - Architecture risk forecasting
 - Release health scoring
 - Organizational bottleneck prediction
 - Multi-quarter engineering intelligence reporting
-

15. Final Architectural Positioning

v1.0 Workflow Intelligence



v2.0 Trustworthy Intelligence



v3.0 Case Intelligence



v4.0 Knowledge Governance



v5.0 Engineering Digital Twin

Version 4.0 transforms the platform from a defect analysis system into a continuously evolving engineering knowledge ecosystem.